

```
/**
 *
=====
=====
 * نسخة "مربوطة - tool-orchestrator-v2.ts
 (Error يرمي TODO لا يوجد) "فعلياً"
 *
=====
=====
 *
 * التثبيتات المطلوبة قبل التشغيل:
 *   npm add docx exceljs pptxgenjs pdfkit
 pdf-parse cheerio pg
 *   npm add -D @types/pdfkit @types/pg
 *
 * متغيرات البيئة المطلوبة:
 *   DATABASE_URL      - رابط PostgreSQL
 (نفس قاعدتك الحالية)
 *   BRAVE_API_KEY     - مفتاح Brave Search
 مجاني حق 2000 طلب/شهر) API
 (https://brave.com/search/api/)
 *   لو مفيش مفتاح
 هيرجع خطأ واضح بدل ما يفشل بصمت web_search
 *
 * من مشروعك تحديداً، (hook) "أدوات محتاجة" خطاب
 لأنني مش عارف تفاصيل تخزين
 *   انظر) ctx.extras ملفاتك الفعلي - مُمَزَّرَة عبر
 (التعليق فوق كل واحدة منها):
 *   - extract_pdf_text /
 summarize_document / speech_to_text: محتاجين
```

```

*      ctx.extras.getFileBuffer(file_id) =>
Promise<Buffer>
*      - github_repos / github_issues: محتاجين
ctx.extras.githubToken
*
*      create_docx, create_xlsx,
create_pptx, create_pdf,
*      execute_code, web_search, fetch_webpage,
run_sql_query, save_memory,
*      recall_memory, semantic_search,
set_reminder, manage_project)
*      شغالة فعلياً بمجرد تثبيت الحزم وضبط
DATABASE_URL.
*
=====
=====

*/

import { Document, Packer, Paragraph,
HeadingLevel } from "docx";
import ExcelJS from "exceljs";
import PptxGenJS from "pptxgenjs";
import PDFDocument from "pdfkit";
import pdfParse from "pdf-parse";
import * as cheerio from "cheerio";
import { Pool } from "pg";

//
=====
=====
// اتصال قاعدة البيانات المشترك - يُستخدم من عدة (0)
أدوات هنا
//
=====

```

=====

```
let _pool: Pool | null = null;
function getPool(): Pool {
  if (!_pool) {
    if (!process.env.DATABASE_URL) {
      throw new Error("DATABASE_URL غير مضبوط في متغيرات البيئة");
    }
    _pool = new Pool({ connectionString:
process.env.DATABASE_URL });
  }
  return _pool;
}

/**
 * migrations لجدول الأدوات دي (شغلها مرة واحدة):
 *
 * CREATE TABLE IF NOT EXISTS user_memories
 (
 *   id SERIAL PRIMARY KEY, user_id TEXT
NOT NULL, fact TEXT NOT NULL,
 *   created_at TIMESTAMPTZ DEFAULT now()
 * );
 * CREATE TABLE IF NOT EXISTS reminders (
 *   id SERIAL PRIMARY KEY, user_id TEXT
NOT NULL, message TEXT NOT NULL,
 *   remind_at TIMESTAMPTZ NOT NULL, fired
BOOLEAN DEFAULT false,
 *   created_at TIMESTAMPTZ DEFAULT now()
 * );
 * CREATE TABLE IF NOT EXISTS projects (
 *   id SERIAL PRIMARY KEY, user_id TEXT
```

```

NOT NULL, name TEXT NOT NULL,
*   is_active BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT now()
* );
*/

//
=====
=====
// 1) نظام تسجيل الأدوات (Plugin Registry)
//
=====
=====

export interface ToolDefinition {
  name: string;
  description: string;
  parameters: Record<string, any>;
  handler: (ctx: ToolContext, args:
Record<string, any>) => Promise<any>;
}

class ToolRegistry {
  private tools = new Map<string,
ToolDefinition>();
  register(tool: ToolDefinition) {
    if (this.tools.has(tool.name))
log("warn", `تم استبدال أداة مسجلة مسبقاً:
${tool.name}`);
    this.tools.set(tool.name, tool);
  }
  get(name: string) {
    return this.tools.get(name);
  }
}

```

```

    getSchemas() {
        return
        Array.from(this.tools.values()).map(({
            name, description, parameters }) => ({
            name, description, parameters }));
    }
    list() {
        return Array.from(this.tools.keys());
    }
}
export const registry = new ToolRegistry();

//
=====
=====
// 2) السجلّ (Logging)
//
=====
=====

type LogLevel = "info" | "warn" | "error";
interface LogEntry { timestamp: string;
level: LogLevel; message: string; meta?:
Record<string, any>; }
const logBuffer: LogEntry[] = [];
const MAX_LOG_ENTRIES = 500;

function log(level: LogLevel, message:
string, meta?: Record<string, any>) {
    const entry: LogEntry = { timestamp: new
Date().toISOString(), level, message, meta
};
    logBuffer.push(entry);
    if (logBuffer.length > MAX_LOG_ENTRIES)

```

```

logBuffer.shift();
    console.log(`[${entry.timestamp}]
[${level.toUpperCase()}] ${message}`, meta
? sanitizeMeta(meta) : "");
}
function sanitizeMeta(meta: Record<string,
any>) {
    const clean = { ...meta };
    for (const key of Object.keys(clean)) if
(/token|secret|key|password/i.test(key))
clean[key] = "[محبوب]";
    return clean;
}
export function getRecentLogs(limit = 100)
{
    return logBuffer.slice(-limit);
}

//
=====
=====
// 3) واجهة موحدة لاستدعاء النموذج (Cloudflare)
//
=====
=====

export interface ToolContext {
    userId: string;
    accountId: string;
    apiToken: string;
    provider?: "cloudflare" | "custom";
    extras?: {
        getFileBuffer?: (fileId: string) =>
Promise<Buffer>;

```

```

    githubToken?: string;
    [key: string]: any;
  };
}

async function fetchWithRetry(url: string,
options: RequestInit, maxRetries = 3):
Promise<Response> {
  let lastError: any;
  for (let attempt = 1; attempt <=
maxRetries; attempt++) {
    try {
      const res = await fetch(url,
options);
      if (res.ok) return res;
      if (res.status === 429 || res.status
>= 500) lastError = new Error(`HTTP
${res.status}: ${await res.text()}`);
      else throw new Error(`HTTP
${res.status}: ${await res.text()}`);
    } catch (err) {
      lastError = err;
    }
    const delayMs = Math.min(1000 * 2 **
(attempt - 1), 8000);
    log("warn", `محاولة
${attempt}/${maxRetries} بعد فشل، إعادة المحاولة بعد
${delayMs}ms`, { url });
    await new Promise((r) => setTimeout(r,
delayMs));
  }
  throw lastError;
}

```

```

async function callCloudflareModel(ctx:
ToolContext, model: string, body:
Record<string, unknown>): Promise<any> {
    const res = await fetchWithRetry(

`https://api.cloudflare.com/client/v4/accou
nts/${ctx.accountId}/ai/run/${model}`,
    { method: "POST", headers: {
Authorization: `Bearer ${ctx.apiToken}`,
"Content-Type": "application/json" }, body:
JSON.stringify(body) }
    );
    return res.json();
}

/** دالة نصية بسيطة تُستخدم داخلياً (مثلاً لتلخيص مستند
بعد استخراج نصه) */
async function callTextModel(ctx:
ToolContext, prompt: string):
Promise<string> {
    const res: any = await
callCloudflareModel(ctx, "@cf/meta/llama-
3.1-8b-instruct", {
        messages: [{ role: "user", content:
prompt }],
    });
    return res?.result?.response ||
res?.response || "";
}

//
=====
=====
// 4) Tool Calling النماذج الداعمة لـ

```

```
//
=====

=====

export const TOOL_CAPABLE_MODELS = [
  "@cf/meta/llama-3.3-70b-instruct-fp8-fast",
  "@cf/meta/llama-3.1-8b-instruct",
  "@cf/meta/llama-3.1-70b-instruct",
  "@cf/openai/gpt-oss-120b",
  "@cf/openai/gpt-oss-20b",
];
export function modelSupportsTools(model:
string): boolean {
  return
  TOOL_CAPABLE_MODELS.includes(model);
}

//
=====

=====

// تسجيل الأدوات - بتنفيذ حقيقي (5)
//
=====

=====

// --- فقط، شغالة زي CF اللي محتاجة نماذج الوسائط ---
// --- (ما هي) ---

registry.register({
  name: "generate_image",
  description: "توليد صورة جديدة من وصف نصي.",
  parameters: { type: "object", properties:
{ prompt: { type: "string" }, model: {
```

```

type: "string" } }, required: ["prompt"] },
  handler: async (ctx, args) => {
    const model = args.model || "@cf/black-forest-labs/flux-1-schnell";
    return { image: await
callCloudflareModel(ctx, model, { prompt:
args.prompt }) });
  },
});

registry.register({
  name: "analyze_image",
  description: "تحليل أو وصف صورة قام المستخدم",
  برفعها.",
  parameters: { type: "object", properties:
{ image_url: { type: "string" }, question:
{ type: "string" } }, required:
["image_url"] },
  handler: async (ctx, args) => ({
    description: await
callCloudflareModel(ctx, "@cf/meta/llama-3.2-11b-vision-instruct", { image:
args.image_url, prompt: args.question ||
"صف هذه الصورة بالتفصيل" }),
  }),
});

registry.register({
  name: "text_to_speech",
  description: "تحويل نص إلى ملف صوتي منطوق.",
  parameters: { type: "object", properties:
{ text: { type: "string" }, voice: { type:
"string" } }, required: ["text"] },
  handler: async (ctx, args) =>

```

```
callCloudflareModel(ctx,
"@cf/deepgram/aura-1", { text: args.text
}),
});

registry.register({
  name: "speech_to_text",
  description: "تحويل تسجيل صوتي مرفوع إلى نص",
  parameters: { type: "object", properties:
{ audio_file_id: { type: "string" } },
required: ["audio_file_id"] },
  handler: async (ctx, args) => {
    if (!ctx.extras?.getFileBuffer) throw
new Error("speech_to_text: لازم تمرر
ctx.extras.getFileBuffer عند بناء
ToolContext");
    const buffer = await
ctx.extras.getFileBuffer(args.audio_file_id
);
    return callCloudflareModel(ctx,
"@cf/openai/whisper-large-v3-turbo", {
audio: Array.from(buffer) });
  },
});

registry.register({
  name: "edit_image",
  description: "تعديل صورة موجودة بناء على تعليمات",
  parameters: { type: "object", properties:
{ image_url: { type: "string" },
instruction: { type: "string" } },
required: ["image_url", "instruction"] },
```

```

handler: async () => {
    // مفعل على حسابات img2img لا يوجد نموذج
    Cloudflare Workers AI حالياً بشكل قياسي.
    // API لو محتاجها فعلاً، أقرب حل عملي هو استدعاء
    من هنا (Replicate/Stability) خارجي
    يحتاج تفعيل
    throw new Error("edit_image: (Replicate/Stability) -
    نموذج img2img خارجي -
    أضف مفتاحه وأنا أربطه");
},
});

// --- الكود ---

registry.register({
    name: "execute_code",
    description: "Python أو JavaScript تنفيذ كود",
    parameters: { type: "object", properties:
    { language: { type: "string", enum:
    ["python", "javascript"] }, code: { type:
    "string" } }, required: ["language",
    "code"] },
    handler: async (_ctx, args) => {
        // Piston API - تنفيذ حقيقي في
        مجاني، بدون مفتاح
        const languageMap: Record<string, {
        language: string; version: string }> = {
            python: { language: "python",
            version: "3.10.0" },
            javascript: { language: "javascript",
            version: "18.15.0" },
        };
        const target =
        languageMap[args.language];

```

```

    if (!target) throw new Error(`لغة غير مدعومة: ${args.language}`);

    const res = await
fetch("https://emkc.org/api/v2/piston/execute", {
    method: "POST",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify({
        language: target.language,
        version: target.version,
        files: [{ content: args.code }],
    }),
});
    if (!res.ok) throw new Error(`فشل تنفيذ الكود: HTTP ${res.status}`);
    const data: any = await res.json();
    return { stdout: data.run?.stdout ||
"", stderr: data.run?.stderr || "",
exitCode: data.run?.code };
},
});

registry.register({
    name: "review_code",
    description: "مراجعة كود موجود واقتراح تحسينات",
    parameters: { type: "object", properties:
{ code: { type: "string" }, language: {
type: "string" } }, required: ["code"] },
    handler: async (ctx, args) =>
        callCloudflareModel(ctx,
"@cf/qwen/qwen2.5-coder-32b-instruct", {

```

```

        messages: [{ role: "user", content:
        `واذكر الأخطاء والتحسينات ( "${args.language} || "غير
        محدد"محدد
        الممكنة:\n\n${args.code}` }],
        }),
    });

registry.register({
    name: "run_sql_query",
    description: "للقراءة فقط على SQL تنفيذ استعلام",
    parameters: { type: "object", properties:
    { query: { type: "string" } }, required:
    ["query"] },
    handler: async (_ctx, args) => {
        const q = String(args.query).trim();
        if (!/^select\s/i.test(q)) throw new
        Error("run_sql_query: مسموح فقط باستعلامات
        SELECT لأسباب أمنية");
        if (/;\s*\S/.test(q.replace(/;\s*$/,
        ""))) throw new Error("run_sql_query: غير
        statement مسموح بأكثر من
        واحد");
        const result = await
        getPool().query(q);
        return { rows: result.rows, rowCount:
        result.rowCount };
    },
});

// --- تنفيذ فعلي عبر الملفات والمستندات
docx/exceljs/pptxgenjs/pdfkit) ---

registry.register({
    name: "create_pdf",

```

```
description: "تحويل نص أو ماركداون إلى ملف  
PDF. base64. يرجع",  
parameters: { type: "object", properties:  
  { title: { type: "string" },  
    content_markdown: { type: "string" } },  
required: ["content_markdown"] },  
handler: async (_ctx, args) => {  
  return new Promise((resolve, reject) => {  
    const doc = new PDFDocument({ margin:  
50 });  
    const chunks: Buffer[] = [];  
    doc.on("data", (c) =>  
chunks.push(c));  
    doc.on("end", () => resolve({ base64:  
Buffer.concat(chunks).toString("base64"),  
mimeType: "application/pdf" }));  
    doc.on("error", reject);  
  
    if (args.title)  
doc.fontSize(20).text(args.title, {  
underline: true }).moveDown();  
    const lines =  
String(args.content_markdown).split("\n");  
    for (const line of lines) {  
      if (line.startsWith("# "))  
doc.fontSize(18).text(line.slice(2)).moveDo  
wn(0.5);  
      else if (line.startsWith("## "))  
doc.fontSize(15).text(line.slice(3)).moveDo  
wn(0.3);  
      else if (line.trim() === "")  
doc.moveDown(0.5);  
      else doc.fontSize(11).text(line);
```

```

    }
    doc.end();
  });
},
});

registry.register({
  name: "extract_pdf_text",
  description: "استخراج النص من ملف PDF مرفوع.",
  parameters: { type: "object", properties:
{ file_id: { type: "string" } }, required:
["file_id"] },
  handler: async (ctx, args) => {
    if (!ctx.extras?.getFileBuffer) throw
new Error("extract_pdf_text: لازم تمرر
ctx.extras.getFileBuffer عند بناء
ToolContext");
    const buffer = await
ctx.extras.getFileBuffer(args.file_id);
    const parsed = await pdfParse(buffer);
    return { text: parsed.text, numPages:
parsed.numpages };
  },
});

registry.register({
  name: "summarize_document",
  description: "إلى نقاط (PDF) تلخيص مستند طويل.",
  parameters: { type: "object", properties:
{ file_id: { type: "string" }, max_points:
{ type: "number" } }, required: ["file_id"]
},

```

```

    handler: async (ctx, args) => {
      const extractTool =
registry.get("extract_pdf_text")!;
      const { text } = await
extractTool.handler(ctx, { file_id:
args.file_id });
      const maxPoints = args.max_points || 5;
      const truncated = text.slice(0, 12000);
      // حماية من تجاوز حد النموذج
      const summary = await
callTextModel(ctx, `لخص هذا المستند في
${maxPoints} نقاط رئيسية
بالعربية:\n\n${truncated}`);
      return { summary };
    },
  });

registry.register({
  name: "create_docx",
  description: "يرجع Word إنشاء ملف
base64.",
  parameters: {
    type: "object",
    properties: { title: { type: "string"
}, sections: { type: "array", items: {
type: "object", properties: { heading: {
type: "string" }, text: { type: "string" }
} } } },
    required: ["title", "sections"],
  },
  handler: async (_ctx, args) => {
    const children: Paragraph[] = [new
Paragraph({ text: args.title, heading:
HeadingLevel.TITLE })];

```

```

    for (const section of args.sections ||
[]) {
      if (section.heading)
children.push(new Paragraph({ text:
section.heading, heading:
HeadingLevel.HEADING_1 }));
      if (section.text) children.push(new
Paragraph({ text: section.text }));
    }
    const doc = new Document({ sections: [{
children }] });
    const buffer = await
Packer.toBuffer(doc);
    return { base64:
buffer.toString("base64"), mimeType:
"application/vnd.openxmlformats-
officedocument.wordprocessingml.document"
};
  },
});

registry.register({
  name: "create_xlsx",
  description: "فعلي. يرجع Excel إنشاء ملف
base64.",
  parameters: {
    type: "object",
    properties: { sheet_name: { type:
"string" }, headers: { type: "array",
items: { type: "string" } }, rows: { type:
"array", items: { type: "array" } } },
    required: ["headers", "rows"],
  },
  handler: async (_ctx, args) => {

```

```

    const workbook = new
ExcelJS.Workbook();
    const sheet =
workbook.addWorksheet(args.sheet_name ||
"Sheet1");
    sheet.addRow(args.headers);
    sheet.getRow(1).font = { bold: true };
    for (const row of args.rows || [])
sheet.addRow(row);
    sheet.columns.forEach((col) =>
(col.width = 18));
    const buffer = await
workbook.xlsx.writeBuffer();
    return { base64:
Buffer.from(buffer).toString("base64"),
mimeType: "application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet" };
  },
});

registry.register({
  name: "create_pptx",
  description: "إنشاء عرض تقديمي PowerPoint
base64. فعلي. يرجع",
  parameters: {
    type: "object",
    properties: { title: { type: "string"
}, slides: { type: "array", items: { type:
"object", properties: { title: { type:
"string" }, bullets: { type: "array",
items: { type: "string" } } } } } },
    required: ["title", "slides"],
  },
  handler: async (_ctx, args) => {

```

```

    const pptx = new PptxGenJS();
    const titleSlide = pptx.addSlide();
    titleSlide.addText(args.title, { x:
0.5, y: 2, fontSize: 32, bold: true });
    for (const slide of args.slides || [])
    {
        const s = pptx.addSlide();
        s.addText(slide.title || "", { x:
0.5, y: 0.4, fontSize: 24, bold: true });
        s.addText((slide.bullets ||
[]).map((b: string) => ({ text: b, options:
{ bullet: true } })), { x: 0.5, y: 1.2,
fontSize: 16 });
    }
    const data = (await pptx.write({
outputType: "base64" })) as string;
    return { base64: data, mimeType:
"application/vnd.openxmlformats-
officedocument.presentationml.presentation"
};
},
});

registry.register({
    name: "create_artifact",
    description: "عرض كود أو تطبيق تفاعلي في لوحة
الجانبيه Artifacts.",
    parameters: { type: "object", properties:
{ language: { type: "string", enum:
["html", "react", "javascript"] }, code: {
type: "string" }, title: { type: "string" }
}, required: ["language", "code"] },
    handler: async (_ctx, args) => ({
artifact: args }),

```

```

});

// --- البحث والمعرفة ---

registry.register({
  name: "web_search",
  description: "بحث حقيقي في الويب مع إرجاع نتائج ومصادرها.",
  parameters: { type: "object", properties:
    { query: { type: "string" } }, required:
    ["query"] },
  handler: async (_ctx, args) => {
    if (!process.env.BRAVE_API_KEY) throw
    new Error("web_search: BRAVE_API_KEY غير
    مضبوط في متغيرات البيئة");
    const res = await
    fetch(`https://api.search.brave.com/res/v1/
    web/search?
    q=${encodeURIComponent(args.query)}`, {
      headers: { Accept:
        "application/json", "X-Subscription-Token":
        process.env.BRAVE_API_KEY },
    });
    if (!res.ok) throw new
    Error(`web_search: HTTP فشل الطلب
    ${res.status}`);
    const data: any = await res.json();
    const results = (data.web?.results ||
    []).slice(0, 8).map((r: any) => ({ title:
    r.title, url: r.url, snippet: r.description
    }));
    return { results };
  },
});

```

```

registry.register({
  name: "fetch_webpage",
  description: "جلب محتوى صفحة ويب محددة عبر",
  parameters: { type: "object", properties:
{ url: { type: "string" } }, required:
["url"] },
  handler: async (_ctx, args) => {
    const res = await fetch(args.url, {
headers: { "User-Agent": "Mozilla/5.0
(compatible; Yode9Bot/1.0)" } });
    if (!res.ok) throw new
Error(`fetch_webpage: فشل الطلب HTTP
${res.status}`);
    const html = await res.text();
    const $ = cheerio.load(html);
    $("script, style, nav, footer,
noscript").remove();
    const text =
$("body").text().replace(/\s+/g, "
").trim().slice(0, 15000);
    return { text, title: $("title").text()
};
  },
});

registry.register({
  name: "translate_text",
  description: "ترجمة نص من لغة إلى أخرى.",
  parameters: { type: "object", properties:
{ text: { type: "string" }, source_lang: {
type: "string" }, target_lang: { type:
"string" } }, required: ["text",

```

```

"target_lang"] },
  handler: async (ctx, args) =>
    callCloudflareModel(ctx, "@cf/meta/m2m100-
1.2b", { text: args.text, source_lang:
args.source_lang || "auto", target_lang:
args.target_lang }),
});

registry.register({
  name: "generate_embeddings",
  description: "تحويل نص إلى تمثيل رقمي (vector) لأغراض البحث الدلالي",
  parameters: { type: "object", properties:
{ text: { type: "string" } }, required:
["text"] },
  handler: async (ctx, args) =>
    callCloudflareModel(ctx, "@cf/baai/bge-
base-en-v1.5", { text: args.text }),
});

registry.register({
  name: "semantic_search",
  description: "بحث دلالي بسيط في ذاكرة المستخدم". المحفوظة (بحث نصي تقريبي، وليس متجهات كاملة بعد)،
  parameters: { type: "object", properties:
{ query: { type: "string" } }, required:
["query"] },
  handler: async (ctx, args) => {
    // على ILIKE بحث pgvector: نسخة عملية بدون
    الكلمات المفتاحية من الاستعلام
    // للترقية لبحث متجهات كامل: ثبت إضافة
    pgvector في PostgreSQL، خزن embeddings
    // في عمود generate_embeddings من
    vector، ORDER BY واستبدل هذا الاستعلام بـ

```

```

embedding <-> query_embedding.
    const keywords =
String(args.query).split(/\s+/).filter(Boolean).slice(0, 5);
    if (keywords.length === 0) return {
matches: [] };
    const conditions = keywords.map((_, i)
=> `fact ILIKE ${i + 2}`).join(" OR ");
    const values = [ctx.userId,
...keywords.map((k) => `_${k}%`)];
    const result = await
getPool().query(`SELECT fact, created_at
FROM user_memories WHERE user_id = $1 AND
(${conditions}) ORDER BY created_at DESC
LIMIT 10`, values);
    return { matches: result.rows };
},
});

// --- إدارة العمل والذاكرة ---

registry.register({
    name: "manage_project",
    description: "إنشاء أو تعديل أو حذف مساحة عمل (Project).",
    parameters: { type: "object", properties:
{ action: { type: "string", enum:
["create", "switch", "delete", "list"] },
project_name: { type: "string" } },
required: ["action"] },
    handler: async (ctx, args) => {
    const pool = getPool();
    switch (args.action) {
        case "create": {

```

```

        const r = await pool.query("INSERT
        INTO projects (user_id, name) VALUES ($1,
        $2) RETURNING id, name", [ctx.userId,
        args.project_name]);
        return { created: r.rows[0] };
    }
    case "switch": {
        await pool.query("UPDATE projects
        SET is_active = false WHERE user_id = $1",
        [ctx.userId]);
        const r = await pool.query("UPDATE
        projects SET is_active = true WHERE user_id
        = $1 AND name = $2 RETURNING id, name",
        [ctx.userId, args.project_name]);
        if (r.rowCount === 0) throw new
        Error(`لا يوجد مشروع باسم:
        ${args.project_name}`);
        return { active: r.rows[0] };
    }
    case "delete": {
        const r = await pool.query("DELETE
        FROM projects WHERE user_id = $1 AND name =
        $2 RETURNING id", [ctx.userId,
        args.project_name]);
        return { deleted: r.rowCount };
    }
    case "list": {
        const r = await pool.query("SELECT
        id, name, is_active FROM projects WHERE
        user_id = $1 ORDER BY created_at DESC",
        [ctx.userId]);
        return { projects: r.rows };
    }
    default:

```

```

        throw new Error(`إجراء غير معروف:
${args.action}`);
    }
},
});

registry.register({
    name: "save_memory",
    description: "حفظ معلومة دائمة عن المستخدم.",
    parameters: { type: "object", properties:
{ fact: { type: "string" } }, required:
["fact"] },
    handler: async (ctx, args) => {
        const r = await getPool().query("INSERT
INTO user_memories (user_id, fact) VALUES
($1, $2) RETURNING id", [ctx.userId,
args.fact]);
        return { saved: true, id: r.rows[0].id
    };
    },
});

registry.register({
    name: "recall_memory",
    description: "استرجاع معلومات محفوظة سابقاً عن
المستخدم.",
    parameters: { type: "object", properties:
{ query: { type: "string" } }, required: []
    },
    handler: async (ctx, args) => {
        if (!args.query) {
            const r = await
getPool().query("SELECT fact, created_at
FROM user_memories WHERE user_id = $1 ORDER

```

```

BY created_at DESC LIMIT 20",
[ctx.userId]);
    return { memories: r.rows };
}
const semanticTool =
registry.get("semantic_search")!;
    return semanticTool.handler(ctx, {
query: args.query });
},
});

registry.register({
    name: "set_reminder",
    description: "جدولة تذكير أو مهمة للمستخدم في",
    parameters: { type: "object", properties:
{ message: { type: "string" }, remind_at: {
type: "string" } }, required: ["message",
"remind_at"] },
    handler: async (ctx, args) => {
        const r = await getPool().query("INSERT
        INTO reminders (user_id, message,
        remind_at) VALUES ($1, $2, $3) RETURNING
        id", [ctx.userId, args.message,
        args.remind_at]);
        return { scheduled: true, id:
        r.rows[0].id, note: "شغل
        كل دقيقة worker/cron من checkDueReminders()
        "عشان تُرسل فعلياً" };
    },
});

/** كل دقيقة لإرسال cron/worker استدع الدالة دي من
*/ التذكيرات المستحقة

```

```

export async function checkDueReminders():
Promise<Array<{ id: number; userId: string;
message: string }>> {
  const pool = getPool();
  const r = await pool.query("SELECT id,
user_id, message FROM reminders WHERE
remind_at <= now() AND fired = false");
  if (r.rowCount && r.rowCount > 0) {
    const ids = r.rows.map((row) =>
row.id);
    await pool.query("UPDATE reminders SET
fired = true WHERE id = ANY($1)", [ids]);
  }
  return r.rows.map((row) => ({ id: row.id,
userId: row.user_id, message: row.message
}));
}

```

// --- التكمالات الخارجية ---

```

registry.register({
  name: "github_repos",
  description: "جلب قائمة مستودعات GitHub
الخاصة بالمستخدم.",
  parameters: { type: "object", properties:
{}}, required: [] },
  handler: async (ctx) => {
    if (!ctx.extras?.githubToken) throw new
Error("github_repos: لازم تمرر
ctx.extras.githubToken");
    const res = await
fetch("https://api.github.com/user/repos?
per_page=50", {
      headers: { Authorization: `Bearer

```

```

    ${ctx.extras.githubToken}`, Accept:
    "application/vnd.github+json" },
    });
    if (!res.ok) throw new
    Error(`github_repos: فشل الطلب HTTP
    ${res.status}`);
    const data: any = await res.json();
    return { repos: data.map((r: any) => ({
    name: r.full_name, private: r.private, url:
    r.html_url })) };
    },
    });

registry.register({
    name: "github_issues",
    description: "جلب قائمة المشاكل المفتوحة في
    مستودع GitHub محدد.",
    parameters: { type: "object", properties:
    { owner: { type: "string" }, repo: { type:
    "string" } }, required: ["owner", "repo"]
    },
    handler: async (ctx, args) => {
        if (!ctx.extras?.githubToken) throw new
        Error("github_issues: لازم تمرر
        ctx.extras.githubToken");
        const res = await
        fetch(`https://api.github.com/repos/${args.
        owner}/${args.repo}/issues?
        state=open&per_page=50`, {
            headers: { Authorization: `Bearer
            ${ctx.extras.githubToken}`, Accept:
            "application/vnd.github+json" },
        });
        if (!res.ok) throw new

```

```

Error(`github_issues: فشل الطلب HTTP
${res.status}`);
    const data: any = await res.json();
    return { issues: data.map((i: any) =>
({ number: i.number, title: i.title, url:
i.html_url })) };
    },
  });

//
=====
=====
// إدارة سياق المحادثة (6)
//
=====
=====

interface ChatMessage { role: "system" |
"user" | "assistant" | "tool"; content:
string; tool_call_id?: string; name?:
string; }
const MAX_CONTEXT_MESSAGES = 40;
function trimContext(messages:
ChatMessage[]): ChatMessage[] {
    if (messages.length <=
MAX_CONTEXT_MESSAGES) return messages;
    const systemMessages =
messages.filter((m) => m.role ===
"system");
    const rest = messages.filter((m) =>
m.role !== "system");
    return [...systemMessages,
...rest.slice(-1 * (MAX_CONTEXT_MESSAGES -
systemMessages.length))];

```

```

}

//
=====
=====
// حلقة المحادثة مع الأدوات (7)
//
=====
=====

const MAX_TOOL_ITERATIONS = 6;
export interface ChatWithToolsResult {
  finalMessage: string; toolCallsLog: Array<{
    tool: string; args: any; result: any;
    durationMs: number }>; iterations: number;
}

export async function chatWithTools(
  ctx: ToolContext,
  model: string,
  messages: ChatMessage[],
  options?: { onToolCall?: (toolName:
string, args: any) => void }
): Promise<ChatWithToolsResult> {
  if (!modelSupportsTools(model)) throw new
Error(`Tool Calling لا يدعم ${model} النموذج.
التماذج المدعومة: ${TOOL_CAPABLE_MODELS.join(",
")}`);

  const toolCallsLog:
ChatWithToolsResult["toolCallsLog"] = [];
  let currentMessages =
trimContext(messages);
  const toolSchemas =

```

```

registry.getSchemas();

for (let i = 0; i < MAX_TOOL_ITERATIONS;
i++) {
  log("info", `من حلقة ${i + 1} تكرار رقم
الأدوات`, { model });
  const response: any = await
callCloudflareModel(ctx, model, { messages:
currentMessages, tools: toolSchemas });
  const toolCalls =
response?.result?.tool_calls ||
response?.tool_calls;

  if (!toolCalls || toolCalls.length ===
0) {
    const finalText =
response?.result?.response ||
response?.response || "";
    log("info", "انتهت الحلقة برد نهائي بدون
استدعاء أدوات إضافية");
    return { finalMessage: finalText,
toolCallsLog, iterations: i + 1 };
  }

  for (const call of toolCalls) {
    const toolName = call.name;
    const args = typeof call.arguments
=== "string" ? JSON.parse(call.arguments) :
call.arguments;
    const tool = registry.get(toolName);
    options?.onToolCall?.(toolName,
args);
    const startedAt = Date.now();
    let result: any;

```

```

        if (!tool) {
            result = { error: `أداة غير مسجلة:
${toolName}` };
            log("error", `محاولة استدعاء أداة غير
موجودة: ${toolName}`);
        } else {
            try {
                result = await tool.handler(ctx,
args);
                log("info", `نجاح تنفيذ الأداة:
${toolName}`, { args });
            } catch (err: any) {
                result = { error: err.message };
                log("error", `فشل تنفيذ الأداة:
${toolName}`, { error: err.message });
            }
        }
        const durationMs = Date.now() -
startedAt;
        toolCallsLog.push({ tool: toolName,
args, result, durationMs });
        currentMessages.push({ role: "tool",
name: toolName, tool_call_id: call.id ||
toolName, content: JSON.stringify(result)
});
    }
    currentMessages =
trimContext(currentMessages);
}

    log("warn", "تم الوصول للحد الأقصى من تكرارات
("حلقة الأدوات دون رد نهائي");
    return { finalMessage: "تم الوصول للحد الأقصى
.", "من محاولات استخدام الأدوات دون التوصل لرد نهائي

```

```
toolCallsLog, iterations:
MAX_TOOL_ITERATIONS };
}

//
=====
=====
// دوال مساعدة للتصدير (8)
//
=====
=====

export function getToolsStatus(): Array<{
name: string; description: string }> {
    return registry.getSchemas().map((t) =>
({ name: t.name, description: t.description
}));
}
export function getLogs(limit?: number) {
    return getRecentLogs(limit);
}
```